Requirements Elicitation & Software Analysis



CS 330 — Software Engineering · Class 5 · Tuesday, October 20, 2026

Topic 3 — Software Development Phases: requirements elicitation and software analysis

LO1 (Learning Outcome 1) and the groundwork for **LO2** (Learning Outcome 2) · Block 5 starts today

The hour at a glance

Time	What
0:00–0:09	Lesson 1 — Requirements engineering: the seven tasks
0:09–0:13	👁️ Visualize-it — the requirements pipeline in seven boxes
0:13–0:22	Lesson 2 — Elicitation: techniques, questions, classic problems
0:22–0:26	✋ Try-it — a two-minute interview in pairs
0:26–0:36	Lesson 3 — Requirements that can be tested; the **SRS** (Software Requirements Specification)
0:36–0:40	🎯 Fun-quiz — functional, non-functional or constraint?
0:40–0:48	Lesson 4 — Use cases, use-case diagrams and swimlanes
0:48–0:56	Lesson 5 — Domain model, **CRC** (Class–Responsibility–Collaborator) cards, traceability, function points
0:56–1:00	🧘 Zen-break, then 🃏 a flash-card sprint on the modeling terms
1:00–1:10	Break — ten real minutes
1:10–2:50	Studio — Lab 05: interview, **SRS-Lite** (a short Software Requirements Specification), use cases, domain model, traceability, function points
2:50–3:00	Wrap-up — the Block 5 box and the exit ticket
at home	Home Sprint (120′) — finish Block 5 (one hour at most), read, practise

Where today's ideas come back

When	What comes back
Today	Studio — Lab 05 / Block 5: interview, **SRS-Lite** (a short Software Requirements Specification), use cases, domain model, traceability matrix, function-point count
Sun Oct 25	First version of Block 5 tagged block5-draft → written feedback before the Mid-Term
Tue Oct 27	Class 6 — design starts from today's use cases and domain model; coached Studio to finish Block 5
Tue Nov 3	Lab 07 / Block 7 — every verification line becomes a test; every use-case scenario becomes a black-box test case; the empty test column of the matrix fills
Tue Nov 17	Mid-Term — Part B: a function-point count; Part C: verifiable requirements and a fully-dressed use case
Dec 1 · 8 · 15	Finishing workshops — improve Block 5 with the feedback; every change between block5-draft and block5 is a managed change
Sun Dec 20	Block 5 final version (tag block5), like every Block
Tue Jan 19	Final — requirements and analysis questions across all outcomes

LESSON 1

Requirements engineering: the seven tasks

0:00–0:09 · slides 4–12

What a requirement is · the seven tasks · change control · volatility

Example: HabitHub (Team Wren, fictional)



What a requirement is

- **Requirement** — a capability the system must provide, or a condition it must meet, to satisfy a stakeholder's need: *what*, not *how*
- **Stakeholder** — anyone with an interest in the system or affected by it: users, client, operators, regulators, third parties
- **Requirements engineering** — understand the needs, record them so they can be checked, manage their changes for the life of the product
- Why a whole class? A mistake is cheapest to fix while it is still a sentence (the cost-of-change curve of Class 1)
- Phoenix pay system (Auditor General of Canada, 2018): more than 100 pay functions deferred or removed; no end-to-end test before launch
- Reference: **ISO/IEC/IEEE 29148** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 29148), 2018 edition

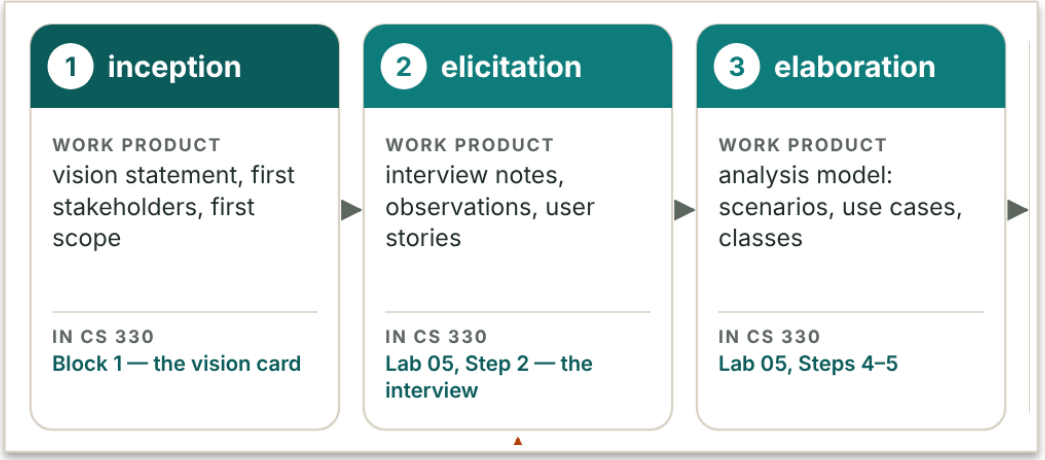
The seven tasks (1/2): inception, elicitation, elaboration

- Pressman and Maxim, chapter 7: seven kinds of work — they overlap and repeat in every iteration
- **1 · Inception** — a first shared understanding: who asks, who uses, what kind of solution

  → vision statement, first stakeholders, first scope
- **2 · Elicitation** — draw the needs out of people: ask, observe, show prototypes

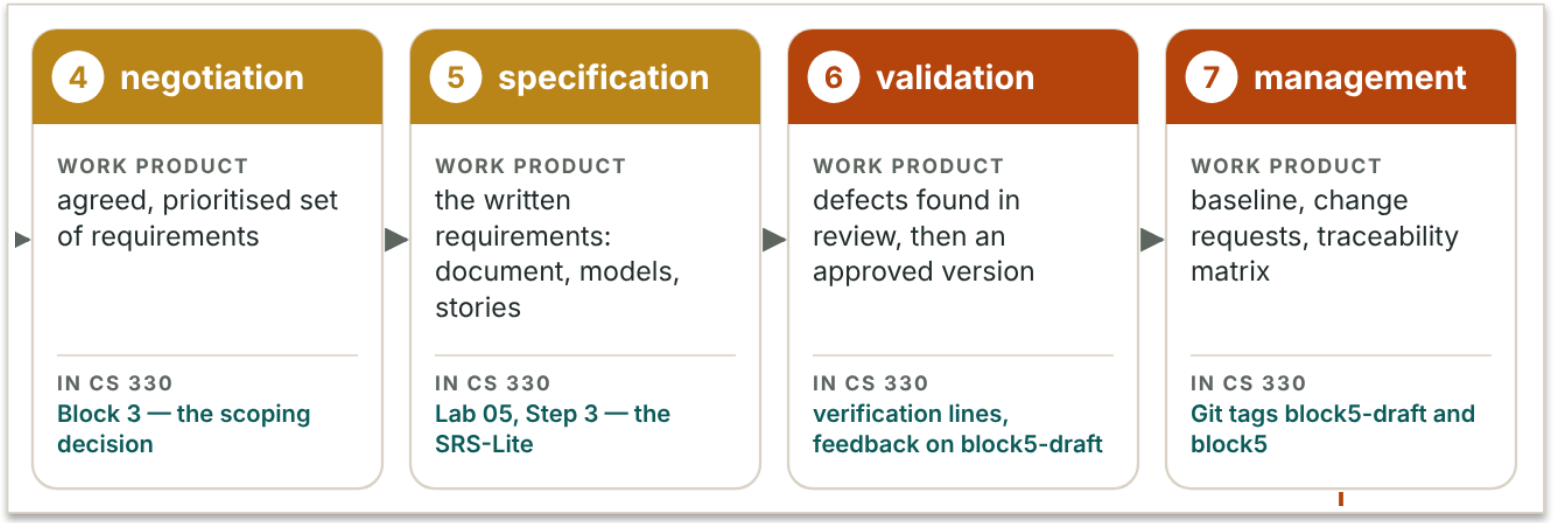
  → interview notes, observations, user stories (Lesson 2)
- **3 · Elaboration** — refine the raw needs into models

  → the analysis model: scenarios, use cases, classes (Lessons 4 and 5)



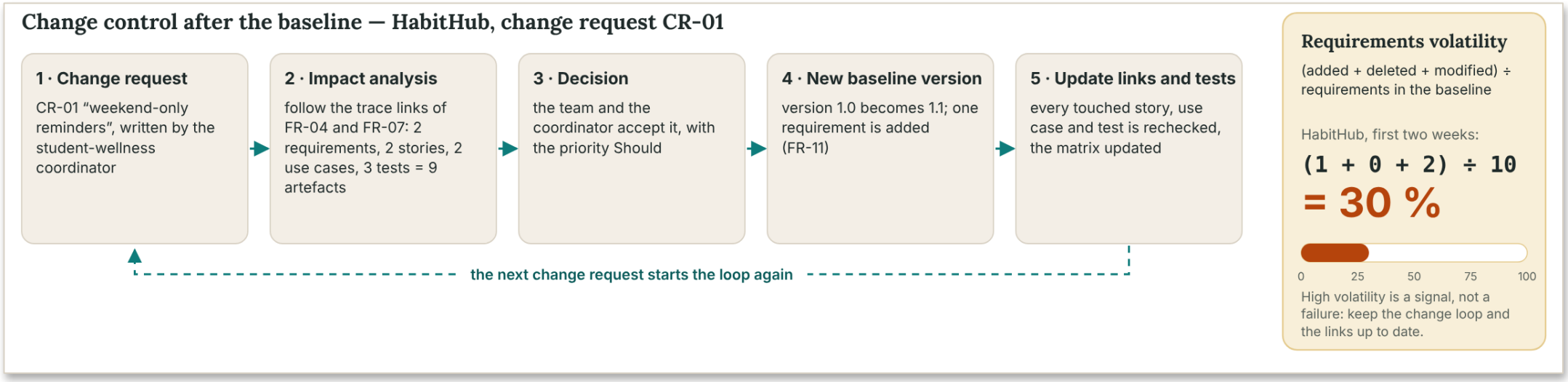
The seven tasks (2/2): negotiation, specification, validation, management

- **4 · Negotiation** — reconcile conflicts; rank by priority, cost and risk → an agreed set
- **5 · Specification** — write it down: document, models, stories, prototype → the **SRS-Lite** (a short Software Requirements Specification)
- **6 · Validation** — check the written requirements: ambiguity, inconsistency, omissions, untestable lines → approved version
- **7 · Management** — identify, control and track changes for the whole life → baseline, change requests, traceability matrix
- Validation examines the requirements; checking the code against them is verification (Class 7)
- No task belongs to one person: the whole team and the stakeholders take part



Requirements change — plan for it

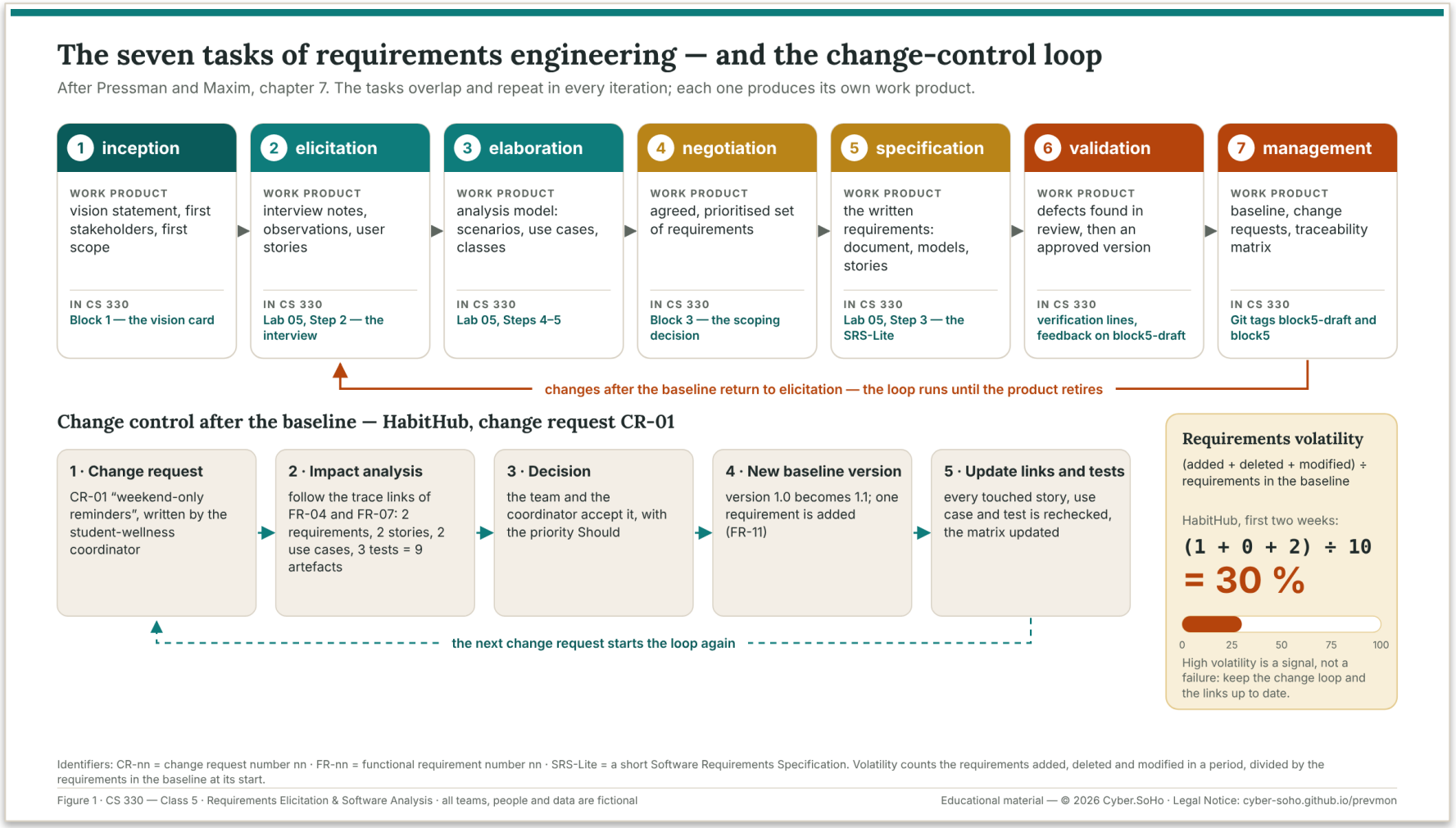
- **Baseline** — a reviewed, agreed version (v1.0) that changes only through change control
- **Change request** — what should change, why, who asks
- **Impact analysis** — everything the change touches, before anyone decides
- **Volatility** = (added + deleted + modified) ÷ requirements in the baseline
- Loop: request → impact analysis → decision → new version → update links and tests
- HabitHub: (1 + 0 + 2) ÷ 10 = **30 %** — a signal to keep the loop running, not a failure



The seven tasks in CS 330

Task	Work product	Where it happens in CS 330
inception	vision statement, first stakeholder list, first scope	Block 1 — the vision card; Class 2 — the stakeholder map
elicitation	interview notes, observations, user stories	Lab 05, Step 2 — the interview of a partner team
elaboration	analysis model: scenarios, use cases, classes	Lab 05, Steps 4 and 5
negotiation	agreed, prioritised set of requirements	Block 3 — the scoping decision; today — the priorities
specification	document, models, user stories, prototype — in CS 330 the SRS-Lite (a short Software Requirements Specification)	Lab 05, Step 3
validation	defects and corrections, then an approved version	verification lines, traceability check, feedback on block5-draft
management	baseline, change requests with decisions, traceability matrix	Git tags block5-draft and block5; changes until December 20

Figure 1 — the seven tasks and the change-control loop



Download it from the module: vector image (.svg) or picture (.png).

Lesson 1 in the module: watch, example, demo

▶️ Watch (15 minutes or shorter)

An introduction to Requirements Engineering — Ian Sommerville, 10:45

Requirements Engineering Processes — Ian Sommerville, 9:12

Requirements engineering challenges — Ian Sommerville, 12:29

🧩 Real-life example

HabitHub: one need through the seven tasks, then one change request

Team Wren (fictional) builds HabitHub: students track daily habits and streaks, with a weekly report. The team follows one stakeholder need — “remind me before I lose my streak” — through the seven tasks of requirements engineering, then handles change request CR-01 (change request number 1), which arrives after the baseline, and measures the requirements volatility of the first two weeks.

💻 Demo program and more

demo1_habithub_requirements_pipeline.py

HabitHub — one need through the seven tasks, then change request CR-01

Figure 1 — The seven tasks of requirements engineering and the change-control loop

Five flash-cards · a formative quiz (4 multiple-choice + 2 multiple-select)

Nothing here is graded.

👁 Visualize-it

The requirements pipeline in seven boxes

1. One and a half minutes, alone, on paper: draw the seven tasks as seven boxes in a row, and one arrow that goes back from management to elicitation — change.
2. Thirty seconds: under each box, write the work product of that task.
3. Two minutes with your team: put a star on the box where your team is today, and a cross on each box you have not started.

The answer is in the notes of this slide and in the module.

LESSON 2

Elicitation

0:13–0:22 · slides 13–21

Stakeholders and tacit knowledge · seven techniques · open, closed and probing questions · three classic problems

Example: StudyBuddy (Team Lynx, fictional)



What elicitation is

- **Elicitation** — draw the needs out of the stakeholders (asking, observing, showing) and record them in their own words
- Requirements are not written down anywhere, ready to be collected
- **Tacit knowledge** — what people use every day but never say, because it seems obvious
- Start with context-free questions: who asked for this, who will use it, what benefit, does a solution already exist?
- Start from the Class 2 stakeholder map; each group has its own **viewpoint**
- Conflicting viewpoints are normal — they are settled in negotiation

Elicitation techniques (1/2): interview, workshop, observation

- Guides list the same core techniques: the **BABOK Guide** (A Guide to the Business Analysis Body of Knowledge) and the **IREB** (International Requirements Engineering Board) syllabus
- **Interview** — structured, semi-structured or unstructured; depth and the reasons behind a need
- **Requirements workshop** — several stakeholders agree together; **JAD** (Joint Application Design), created at **IBM** (International Business Machines) in the late 1970s
- **Observation** — watch real work in its real setting: the main way to reach tacit knowledge
- Trade-offs: interviews cost time per person · workshops need a facilitator · observation is slow

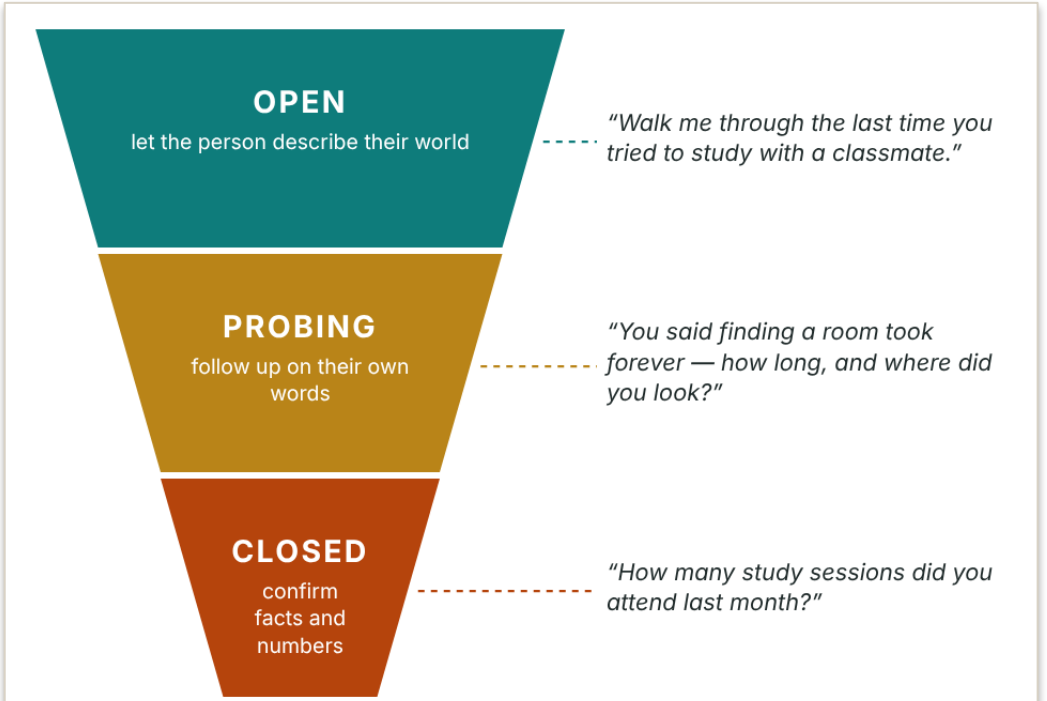
Technique and what it is best for	Depth of answers	People reached	Tacit knowledge	Time needed
interview depth and the reasons behind a need	●●●	●○○	●●○	●●●
requirements workshop agreeing across several viewpoints	●●○	●●○	●●○	●●○
observation real behaviour, knowledge nobody says aloud	●●●	●○○	●●●	●●●
questionnaire counts and priorities across many people	●○○	●●●	●○○	●○○

Elicitation techniques (2/2): questionnaire, prototype, brainstorming, document analysis

Technique	Best for	Weak for
interview	depth, the reasons behind a need	many people; time per person
requirements workshop	agreeing across viewpoints	needs a facilitator and the right people
observation	tacit knowledge, real behaviour	slow; people change while observed
questionnaire	counts and priorities across many people	why: no follow-up possible
prototype	users who cannot describe what they need	can be taken for the finished product
brainstorming	many ideas quickly	ideas still need evaluation
document analysis	existing data, rules, forms	knowledge that was never written down

Asking questions: open, closed, probing — the funnel

- **Open** — invites a description: “Walk me through the last time…”
- **Probing** — follows up on the person's own words: “You said… — how long?”
- **Closed** — confirms a fact: yes or no, a number, a choice
- Funnel order: open → probing → closed
- Ask about the last real occurrence, not the future · do not describe your solution
- No leading questions · listen more than you talk · write exact words · note the surprise



Team Lynx asked two questions of each kind (Lab 05, Step 2).

The three classic problems of elicitation

- Christel and Kang, **SEI** (Software Engineering Institute), 1992 — still true today
- **Scope** — unclear boundary; too little or too much detail → vision, the Block 3 scoping decision, a parked list
- **Understanding** — needs unknown, unspoken or understood differently → several techniques, a glossary
- **Volatility** — needs change → short iterations and change control
- Interview notes are data about people: invented names only; no recording without permission (Legal Notice, sections 6.7, 17, 18.2)

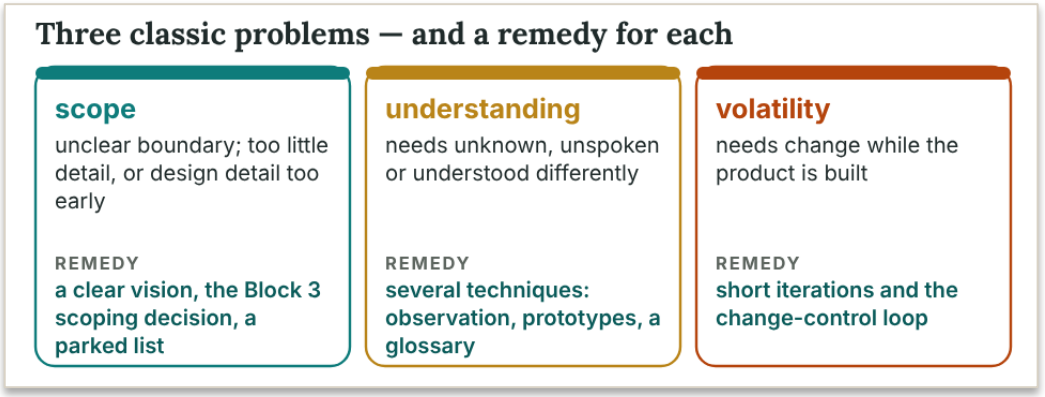
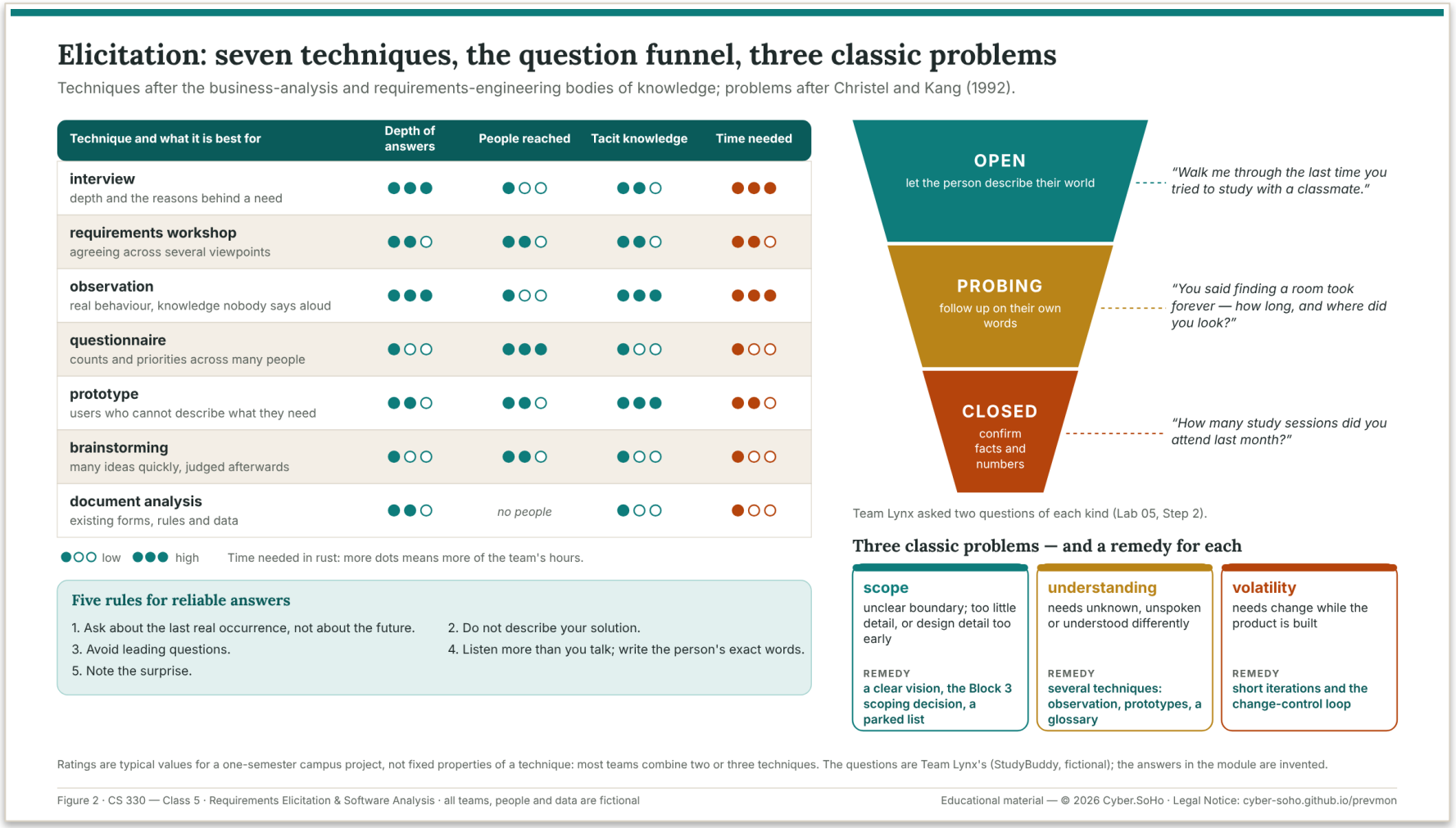


Figure 2 — techniques, the question funnel, the classic problems



Download it from the module: vector image (.svg) or picture (.png).

Lesson 2 in the module: watch, example, demo

▶️ Watch (15 minutes or shorter)

5 Powerful Requirement Gathering Techniques Every Business Analyst Should Know — Bridging the Gap - Resources for Business Analysts, 8:27

Requirements Elicitation – What Questions to Ask — Bridging the Gap - Resources for Business Analysts, 8:57

Elicitation Problems - Georgia Tech - Software Development Process — Udacity, 2:17

Stakeholders, Viewpoints and concerns — Ian Sommerville, 8:07

🧩 Real-life example

StudyBuddy: six questions, three answers, one surprise

Team Lynx (fictional) prepares six interview questions — two open, two closed, two probing — for its partner team, who play the users of StudyBuddy, an app for finding study partners and booking study sessions. After the interview, the team turns the recorded answers into needs, sorts each need as inside or outside the Block 3 scope, and finds the surprise: the one need inside the scope that no user story covers yet.

💻 Demo program and more

demo2_studybuddy_interview.py

StudyBuddy — classify the questions, find the needs and the surprise

Figure 2 — Elicitation techniques and the question funnel

Five flash-cards · a formative quiz (4 multiple-choice + 2 multiple-select)

Nothing here is graded.

✋ Try-it

A two-minute interview in pairs

1. Pair up with someone from another team. Partner A interviews partner B for two minutes: “What annoys you most about booking a study room?” Start with the open question, then ask two probing questions on B's own words.
2. Write down B's exact words and one surprise.
3. Swap: B interviews A for two minutes. In the last ten seconds, each of you says the surprise aloud to the other.

The answer is in the notes of this slide and in the module.

LESSON 3

Requirements that can be tested

0:26–0:36 · slides 22–31

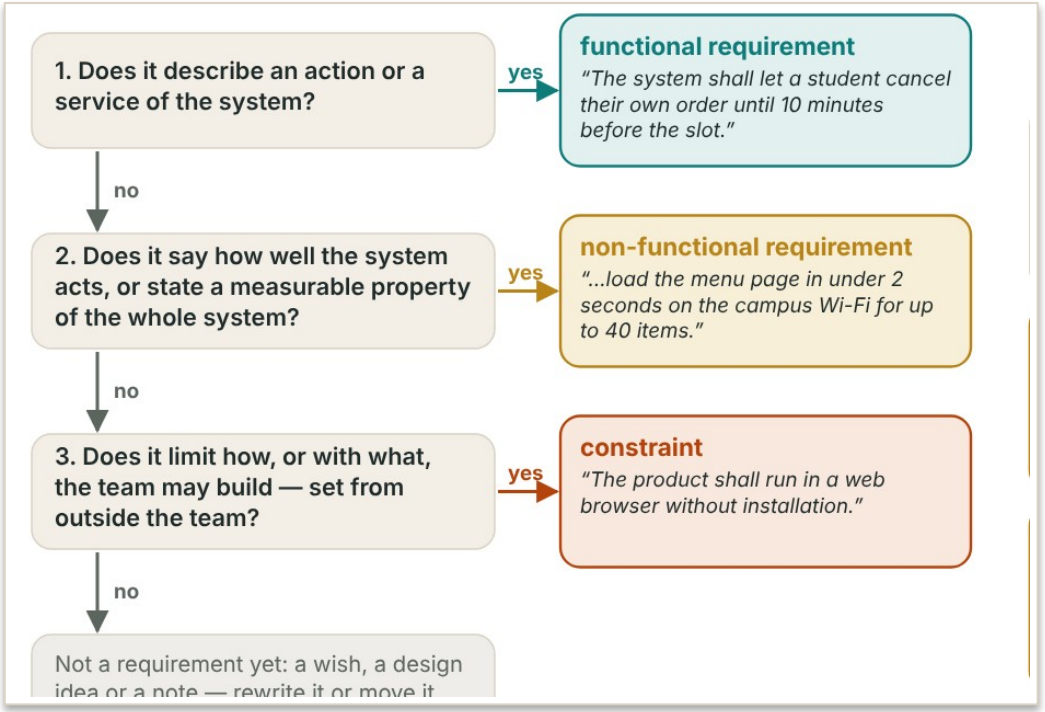
Functional, non-functional, constraint · quality model · six rules · the specification

Example: QueueLess (Team Falcon, fictional)



Three kinds of statements

- **Functional requirement** — a behaviour or service: *the system shall let a student cancel their own order until 10 minutes before the slot*
- **Non-functional requirement** — how well, or a property of the whole system: *the menu page shall load in under 2 seconds on the campus Wi-Fi for up to 40 items*
- **Constraint** — a limit set outside the team: *the product shall run in a web browser without installation*
- Test: an action? → functional · how well? → non-functional · a limit on how or with what? → constraint



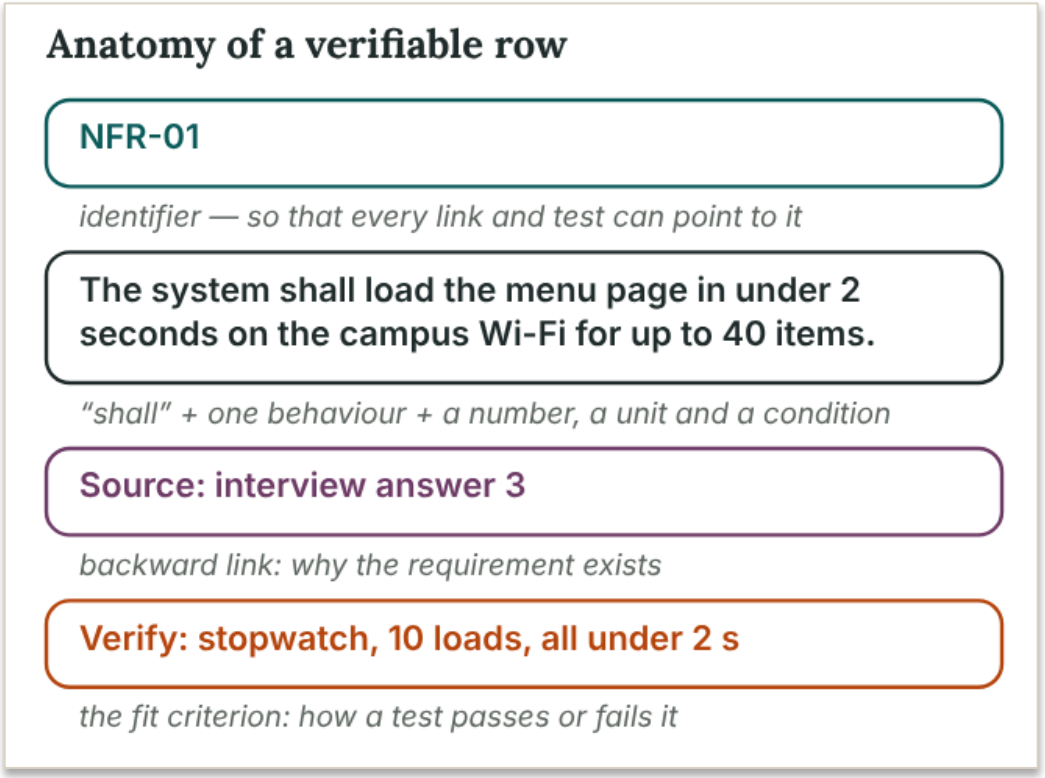
Quality characteristics: the product-quality model

- **ISO/IEC 25010** (International Organization for Standardization / International Electrotechnical Commission standard 25010), 2023 edition — nine characteristics:
- functional suitability · performance efficiency · compatibility
- interaction capability · reliability · security
- maintainability · flexibility · safety (new in 2023)
- Block 5: exactly five non-functional requirements — performance, security or privacy, usability, reliability, maintainability
- A requirement may cite a standard (**WCAG** (Web Content Accessibility Guidelines) 2.2, level AA) or a law (Québec's Law 25): checkable, not negotiable



Requirements that can be tested: six rules

- **Verifiable** — a finite, affordable check exists: test, inspection, analysis or demonstration
- 1 · “shall” for a binding requirement (“should” = a goal)
- 2 · one behaviour per requirement
- 3 · a number, a unit and a condition — or a named check
- 4 · no vague words: fast, easy, user-friendly, intuitive, secure…
- 5 · what, not how — no buttons, no colours
- 6 · an identifier, a source and a verification line



From weak to strong: the QueueLess rewrites

Weak	Strong
“The app should be fast and easy.”	NFR-01 (performance): The system shall load the menu page in under 2 seconds on the campus Wi-Fi for a menu of up to 40 items. Verify: stopwatch, 10 loads, all under 2 seconds.
“Users can cancel.”	FR-05: The system shall let a student cancel their own order until 10 minutes before the chosen pick-up slot; after that, the cancel action shall not be offered. Source: US-04, interview answer 2. Verify: try at −11 minutes and at −9 minutes.
what changed	“should” → “shall” · vague adjectives → a number, a unit and a condition · “users” → a named actor with a limit · every strong line names its test

Two helpers: sentence patterns and the fit criterion

Pattern	Keyword	Template	QueueLess example
ubiquitous	(none)	The <system> shall <response>.	The system shall show the price of each menu item.
event-driven	When	When <trigger>, the <system> shall <response>.	When an order is marked ready, the system shall notify the student.
state-driven	While	While <state>, the <system> shall <response>.	While the kitchen is closed, the system shall refuse new orders.
unwanted behaviour	If … then	If <condition>, then the <system> shall <response>.	If a payment flag is missing, then the system shall keep the order out of the kitchen list.
optional feature	Where	Where <feature is included>, the <system> shall <response>.	Where group orders are enabled, the system shall split the total per student.

The Software Requirements Specification

- **SRS (Software Requirements Specification)** — states the requirements completely, consistently and verifiably
- **IEEE Std 830** (Institute of Electrical and Electronics Engineers Standard 830)-1998 → superseded in 2011 by **ISO/IEC/IEEE 29148** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 29148); current edition 2018; next edition at the draft ballot (October 2026)
- Outline: introduction · references · specific requirements · verification · appendices
- Block 5 = an **SRS-Lite** (a short Software Requirements Specification): seven sections on two pages

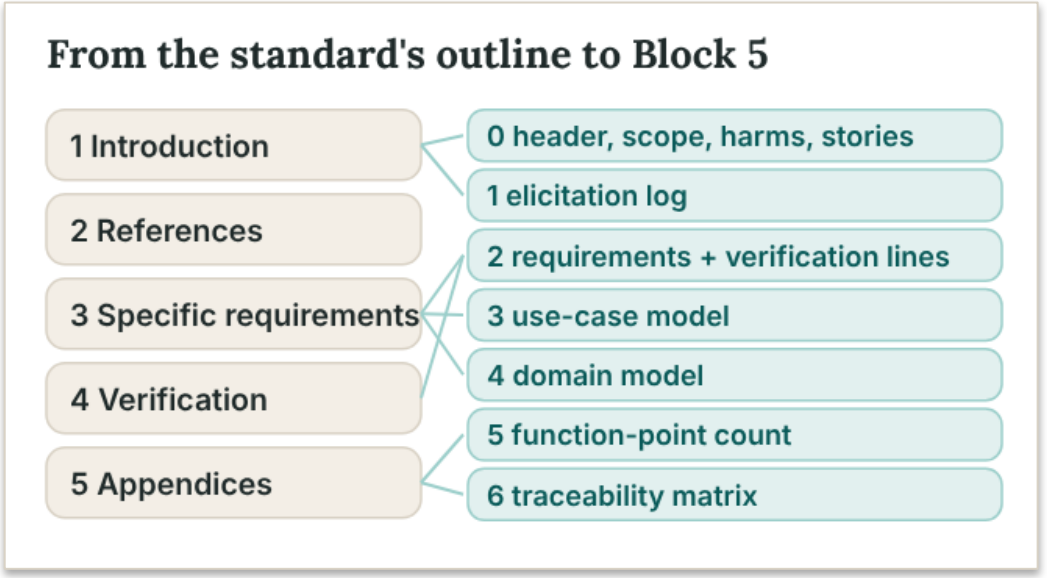
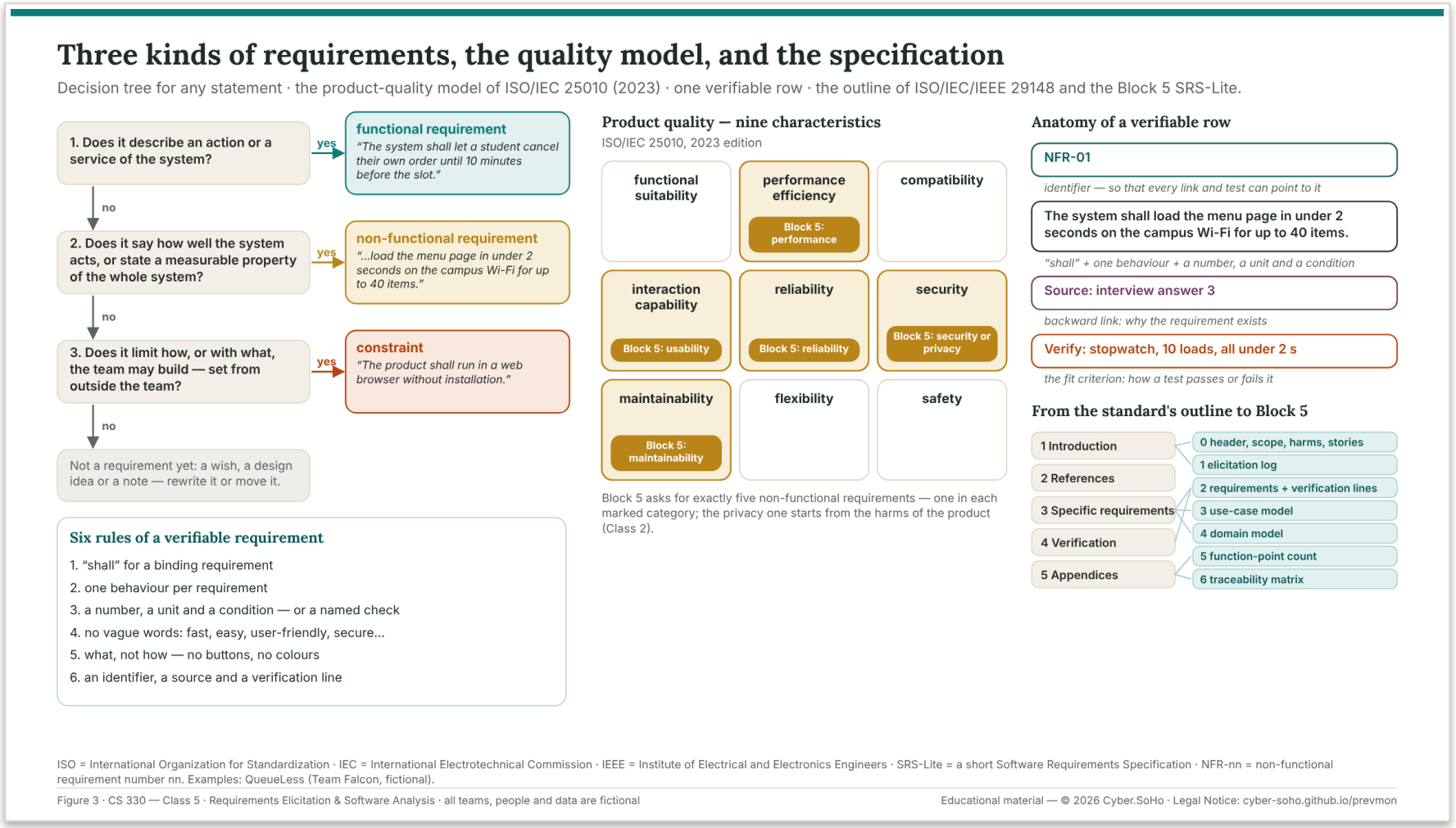


Figure 3 — kinds of requirements, quality model, specification



Download it from the module: vector image (.svg) or picture (.png).

Lesson 3 in the module: watch, example, demo

▶️ Watch (15 minutes or shorter)

What are Non-functional Requirements and How Do They Work? — AltexSoft, 9:29

Easy Approach to Requirements Syntax (EARS) - The Basics — Saxion Media Xpert Centre, 13:33

Requirement Engineering - Frameworks And Standards — Birgit Penzenstadler, 13:21

L07 04 Characteristics of Good Requirements — Phil Koopman, 3:05

🧩 Real-life example

QueueLess: from six drafts to a verifiable set

Team Falcon (fictional) builds QueueLess, a cafeteria pre-order app. It reviews six draft requirements, classifies each one as a functional requirement, a non-functional requirement or a constraint, finds the defects that make some of them impossible to test, and rewrites them in the row format of Block 5: identifier · "The system shall …" · source · how we verify it.

💻 Demo program and more

demo3_queueless_requirement_checker.py

QueueLess — a requirement checker

Figure 3 — Three kinds of requirements, the quality model, and the requirements specification

Five flash-cards · a formative quiz (4 multiple-choice + 2 multiple-select)

Nothing here is graded.

🎯 Fun-quiz

Functional, non-functional or constraint? Three statements

1. Read each statement on the screen and vote with your fingers: one finger for a functional requirement, two for a non-functional requirement, three for a constraint.
2. Statement 1: “The system shall let a student book a study room.”
3. Statement 2: “Pages shall load in under 2 seconds on the campus Wi-Fi.”
4. Statement 3: “The system must be written in Python because it is the department standard.”

The answer is in the notes of this slide and in the module.

LESSON 4

Use cases, use-case diagrams, swimlanes

0:40–0:48 · slides 32–40

Analysis model · actors and scenarios · the fully-dressed use case · «include» and «extend» · swimlanes · flows to tests

Example: ClubLend (Team Owl, fictional)



The analysis model: from words to models

- **Analysis model** — what the system must do, in the terms of the problem domain
- Scenario-based elements: user stories, use cases, activity diagrams (this lesson)
- Class-based elements: classes, attributes, associations, **CRC** (Class–Responsibility–Collaborator) cards (Lesson 5)
- Behavioural elements: states and transitions
- Rule: the users' words; every technical choice waits for the design (Class 6)

Actors, use cases and scenarios

- Ivar Jacobson, **OOPSLA** (Object-Oriented Programming, Systems, Languages and Applications conference) 1987
- **Actor** — a role, not a person; outside the system; primary (has the goal) or supporting
- **Use case** — the interactions by which the system helps a primary actor reach one goal: verb + object
- **Scenario** — one concrete path through a use case
- **Main flow** — the numbered steps in which everything succeeds
- **Alternative flow** — branches at a numbered step when a condition holds: *4a*
- A user story opens the conversation; the use case records what it found

A fully-dressed use case (1/2): the template

Field	What it holds
Identifier and name	UC-nn (use case number nn) and a goal: verb + object
Primary actor / supporting actors	who has the goal / who helps
Goal	one sentence, in the actor's words
Trigger	the event that starts the use case
Preconditions	what must already be true before it starts
Main flow	numbered steps, alternating actor does / system does
Alternative flows	at least two, each starting at a numbered step: 4a. The item is on loan: …
Postconditions	what is true after success, and what stays unchanged after a refusal
Related requirements	the identifiers of the functional and non-functional requirements realised

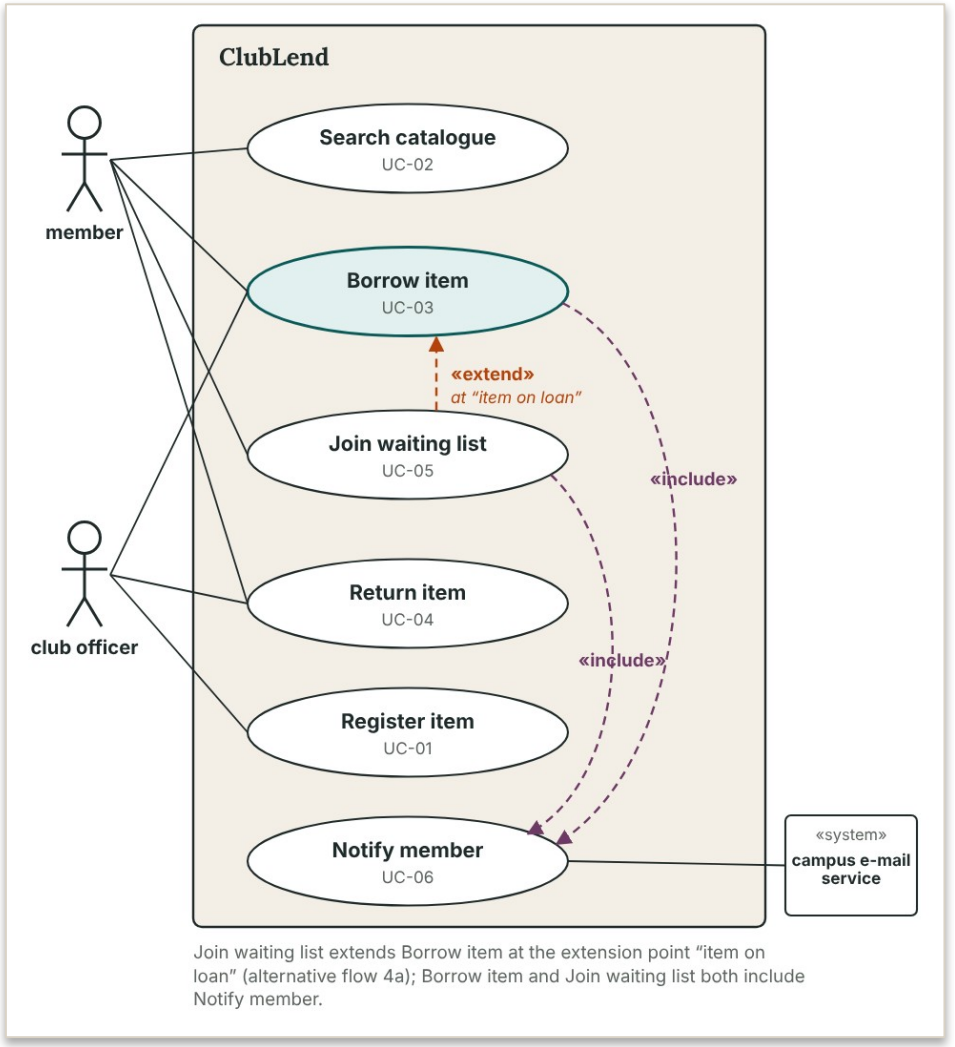
A fully-dressed use case (2/2): ClubLend, UC-03 Borrow item

- 1 · member chooses an item, selects Borrow
- 2 · system checks: no overdue loan → **2a** refused, overdue item shown
- 3 · system checks: fewer than 3 active loans → **3a** refused, limit shown
- 4 · system checks the item is available, reserves it → **4a** waiting list offered
- 5 · system creates the loan, shows a 48-hour pick-up code (include UC-06)
- 6 · club officer hands the item over → **6a** not picked up in 48 hours: loan cancelled
- 7 · loan active, due back in 14 days
- Postconditions · related: FR-02 to FR-05, NFR-02



The use-case diagram: include and extend

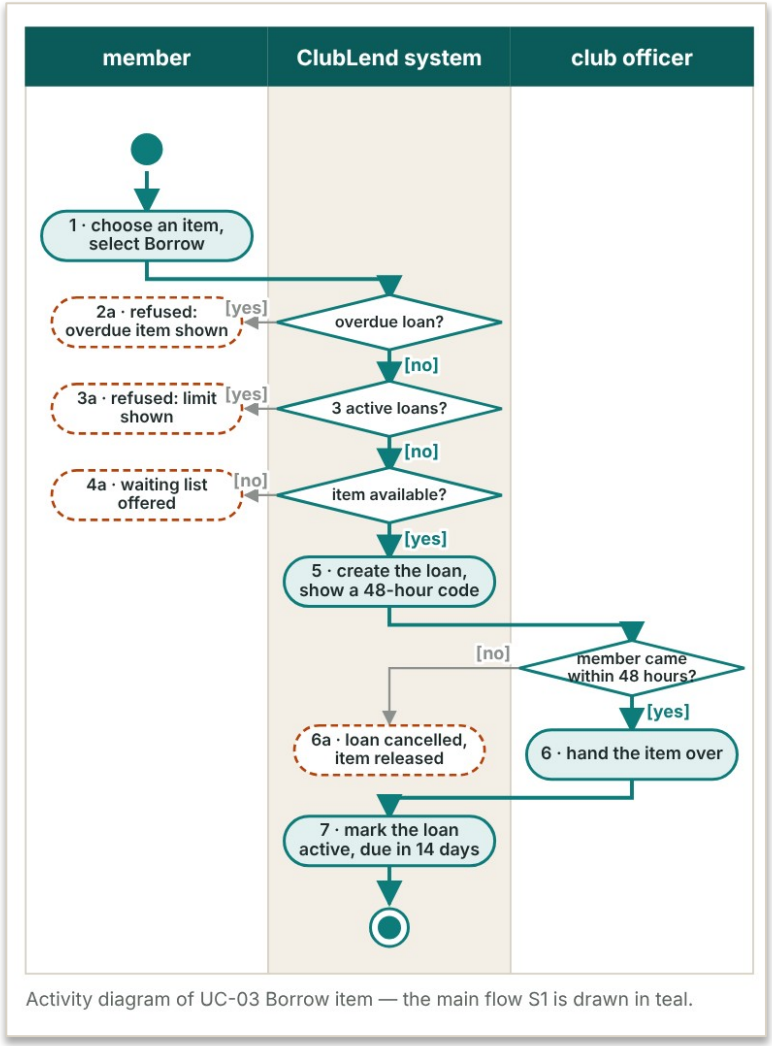
- **UML** (Unified Modeling Language) notation, published by the **OMG** (Object Management Group): version 2.5.1, December 2017
- System boundary · actors outside · use cases inside · associations
- **«include»** — the base always performs it; arrow from the base to the included use case
- **«extend»** — optional behaviour at an extension point, under a condition; arrow to the base
- Use both sparingly: goals and actors first



Join waiting list extends Borrow item at the extension point "item on loan" (alternative flow 4a); Borrow item and Join waiting list both include Notify member.

Activity diagrams with swimlanes

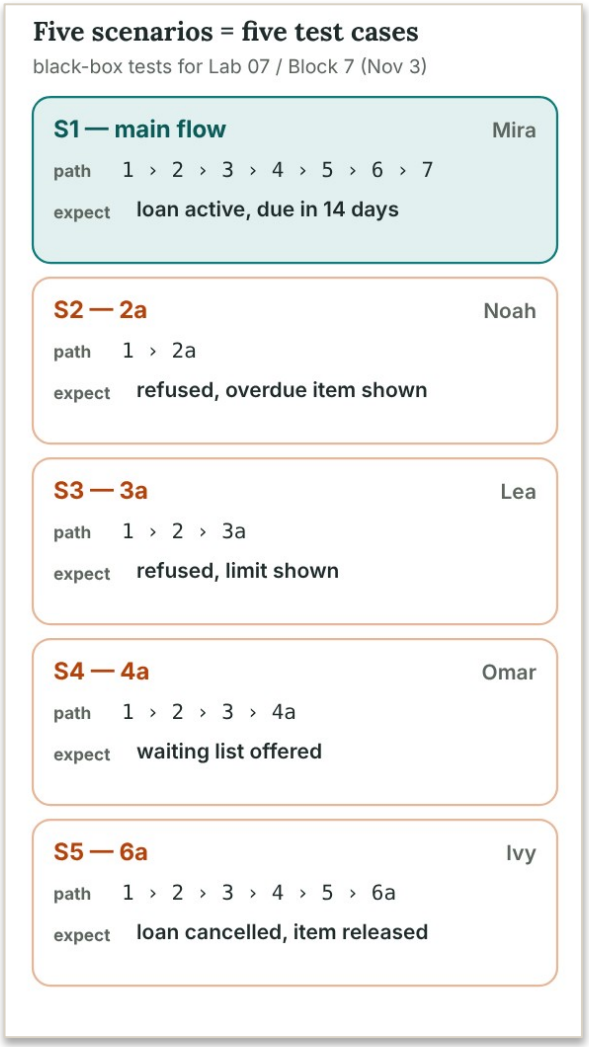
- Initial node · actions · decisions with [guards] · merge · fork and join · final node
- **Swimlane** — one lane per actor or system: who does each step
- UC-03: member | ClubLend system | club officer
- Each diamond = one alternative flow; each path start → end = one scenario
- When text and drawing disagree, one of them is wrong



Activity diagram of UC-03 Borrow item — the main flow S1 is drawn in teal.

From flows to test cases

- Every scenario → at least one **black-box test case**
- UC-03: 1 main flow + 4 alternative flows = 5 test cases
- S1 Mira — main flow · S2 Noah — 2a · S3 Lea — 3a · S4 Omar — 4a · S5 Ivy — 6a
- Lab 07 / Block 7 (November 3) starts from this table; Class 7 adds the boundaries
- A use case with only a main flow gives one test



Lesson 4 in the module: watch, example, demo

▶️ Watch (15 minutes or shorter)

UML (Unified Modeling Language) use case diagrams: Everything you need to know — Lucid Software, 12:42

How to write detailed use case descriptions — Karan Jetli Live, 7:49

UML (Unified Modeling Language) activity diagram tutorial: How to build your own with Lucidchart — Lucid Software, 4:30

Use Case Example - Georgia Tech - Software Development Process — Udacity, 3:08

🧩 Real-life example

ClubLend: one use-case model, one swimlane diagram, five scenarios

Team Owl (fictional) builds ClubLend, the lending app of a student club that lends board games and books: catalogue, loans, returns, waiting list. The team names its actors, draws the use-case diagram (three actors, six use cases, one «include», one «extend»), fully dresses UC-03 Borrow item with four alternative flows, draws its activity diagram in three swimlanes, and lists the five scenarios as the test cases of Block 7.

💻 Demo program and more

demo4_clublend_borrow_usecase.py

ClubLend — walking UC-03 Borrow item through its five scenarios

Figure 4 — The ClubLend use-case model: diagram, swimlanes and scenarios as tests

Five flash-cards · a formative quiz (4 multiple-choice + 2 multiple-select)

Nothing here is graded.

LESSON 5

Domain model, traceability, function points

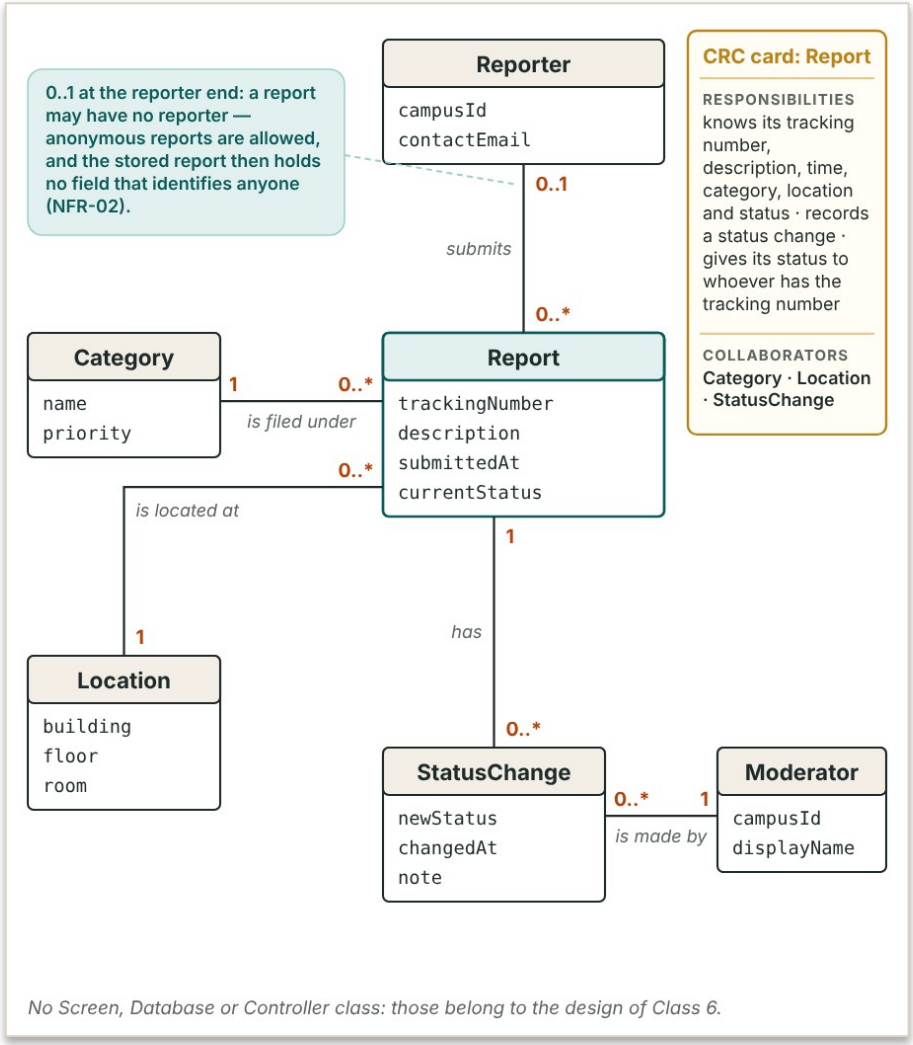
0:48–0:56 · slides 41–50

Domain model · nouns and **CRC** (Class–Responsibility–Collaborator) cards · validation · traceability · function points

Example: SafeCampus (Team Heron, fictional)

The domain model

- **Domain model** — the concepts users talk about and the product must remember; a **UML** (Unified Modeling Language) class diagram without operations or software parts
- **Analysis class · attribute · association** (a verb phrase) · **multiplicity** (1, 0..1, 0..*, 1..*)
- Read one end at a time: a report is filed under exactly **1** category; a category groups **0..*** reports
- SafeCampus: a report is submitted by **0..1** reporter — anonymous reports allowed (NFR-02)



Finding the classes: nouns and CRC (Class–Responsibility–Collaborator) cards

- Underline the nouns of the stories and use cases: a **grammatical parse**
- Keep a noun if the product must remember information about it
- A single value → an attribute · a screen or database → design · two words for one thing → glossary
- Verbs give the associations
- **CRC (Class–Responsibility–Collaborator) card** — class name · responsibilities (knows, does) · collaborators
- Beck and Cunningham, **OOPSLA** (Object-Oriented Programming, Systems, Languages and Applications conference) 1989 — role-play a scenario with the cards; keep five to eight

Validating the requirements and the models

- A **review** with a checklist — your partner team, the feedback on block5-draft
- 1 source · 2 verifiable · 3 unambiguous · 4 consistent
- 5 feasible · 6 modelled (each requirement in a use case, each class used) · 7 in scope
- Questions 1, 6 and 7 can be answered from a table: the traceability matrix

Traceability

- **Traceability** — follow a requirement backward to its source and forward to models, code and tests
- **Traceability matrix** — one row per requirement; an empty cell is a gap
- No source → find one or delete · no use case → model it
- Story without a requirement → write one or park it · no test → Block 7
- SafeCampus: FR-08 → UC-05 written · FR-09 deleted · US-06 parked
- The matrix also drives the impact analysis of every change request

Traceability check — both directions

requirement	user story	use case	element
FR-01	US-01	UC-01	EI
FR-02	US-01	UC-01	EO
FR-03	US-02	UC-02	EQ
FR-04	US-03	UC-03	EI
FR-05	US-02	UC-02	EO
FR-06	US-03	UC-03	EQ
FR-07	US-04	UC-04	EO
FR-08	US-05	UC-05 new	EI
~~FR-09~~	~~— gap —~~	~~— gap —~~	~~EQ~~

FR-08 has a story but no use case
→ the team writes UC-05 "Add hazard category"

FR-09 has no story and no use case
→ nobody asked for it: deleted

US-06 "attach a photo" has no requirement
→ outside the Block 3 scope: parked

Result: 8 requirements, 5 stories, no empty cell.

Function points (1/2): the five elements

Element	What it counts	Weight	SafeCampus example
EI (External Input)	data or a command entering from outside that changes stored data	4	submit a hazard report
EO (External Output)	information leaving with calculated or generated data	5	monthly summary; confirmation with a new tracking number
EQ (External Inquiry)	stored data shown without calculation or change	4	check the status of a report
ILF (Internal Logical File)	a group of related data the system maintains	10	the reports; the categories
EIF (External Interface File)	a group of related data read from another system	7	the campus directory; the building list

Function points (2/2): the count, the factor and its use

- **UFP** (Unadjusted Function Points) = 4 × **EIs** (External Inputs) + 5 × **EOs** (External Outputs) + 4 × **EQs** (External Inquiries) + 10 × **ILFs** (Internal Logical Files) + 7 × **EIFs** (External Interface Files)
- **FP** (Function Points) = **UFP** (Unadjusted Function Points) × (0.65 + 0.01 × ΣF) — 14 factors rated 0 to 5; ΣF = 35 → × 1.00
- Board example: 6, 4, 3, 2, 1 → 24 + 20 + 12 + 20 + 7 = **83**
- SafeCampus: 69 function points × 50 lines = 3,450 lines — against 3,000 in the Class 3 list
- Two independent estimates that agree are worth more than one

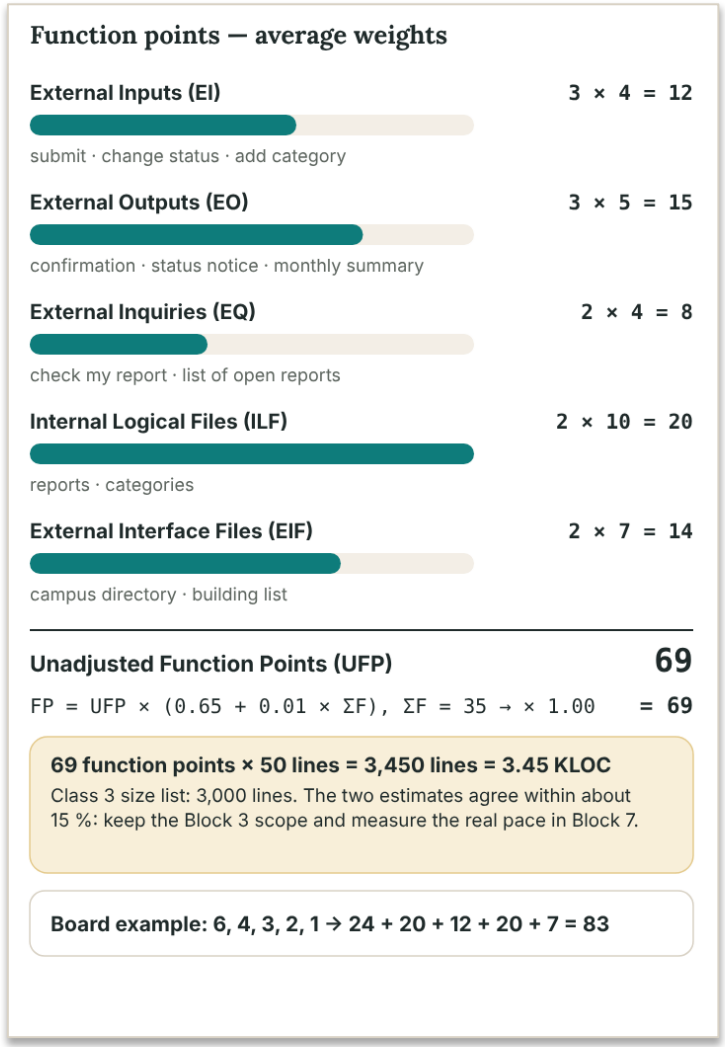
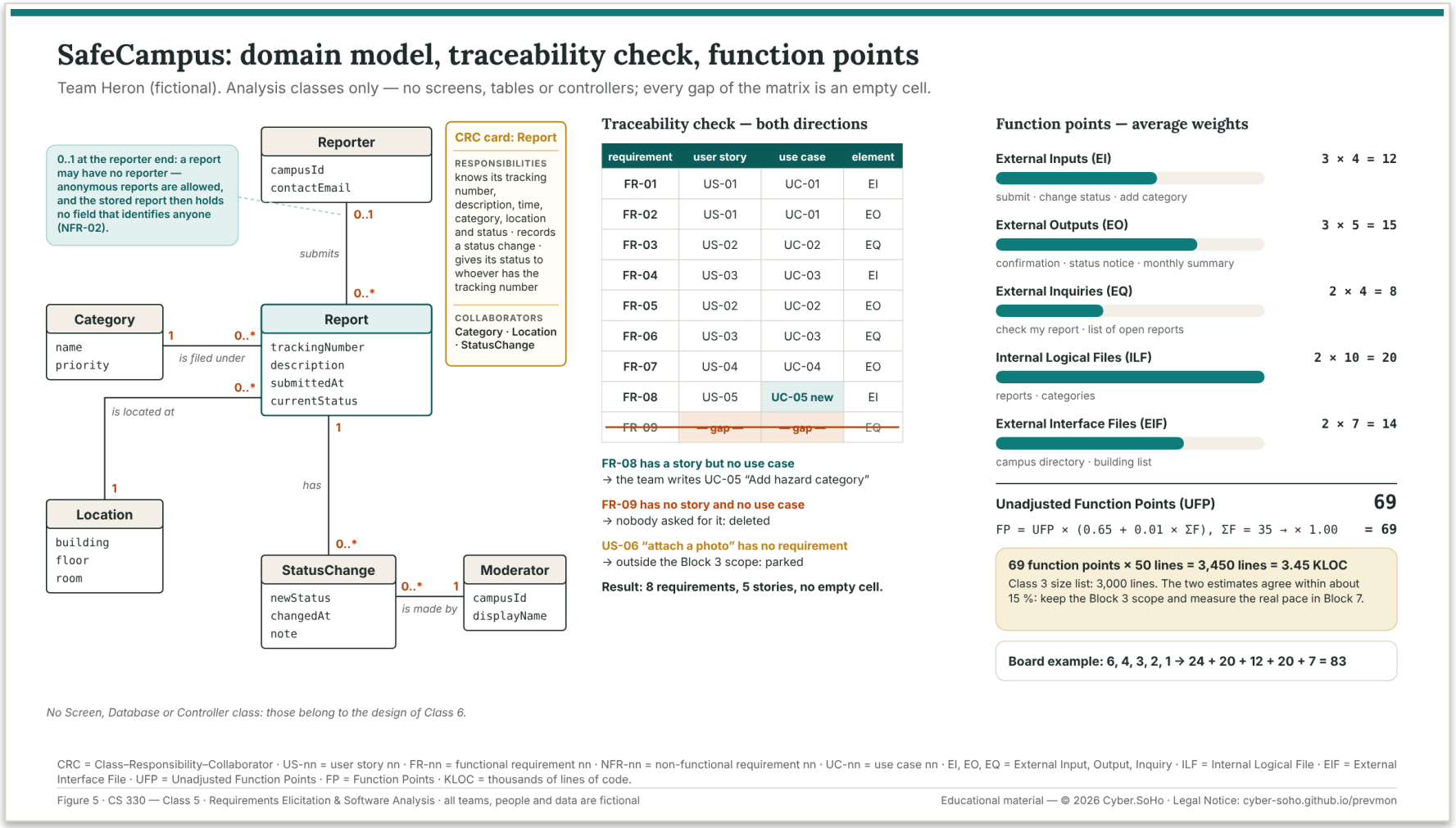


Figure 5 — domain model, traceability, function points



Download it from the module: vector image (.svg) or picture (.png).

Lesson 5 in the module: watch, example, demo

▶️ Watch (15 minutes or shorter)

Domain Modeling Crash Course — Tobias O lsson, 7:50

UML (Unified Modeling Language) class diagrams — Lucid Software, 12:24

Class, Responsibility, Collaborator (CRC) Car d Design — Timothy Steele, 8:12

Function Point - Step by Step Guide with Nu merical Examples — Master2Teach, 10:34

Function Point Analysis - An Overview — Kr upananda Mannekunta, 12:03

🧩 Real-life example

SafeCampus: classes from nouns, a matrix with three gaps, and 69 function points

Team Heron (fictional) builds SafeCampus: a student or a staff member reports a hazard on campus — a broken stair light, a wet floor, a blocked exit — under their name or anonymously, and follows its status until it is fixed. The team finds its analysis classes from the nouns of its stories, checks them with a round of index cards, draws the domain model, checks its traceability matrix in both directions, corrects three gaps, and counts the function points of what remains, to compare with its Class 3 estimate.

💻 Demo program and more

demo5_safecampus_trace_fp.py

SafeCampus — traceability in both directions, then a function-point count

Figure 5 — The SafeCampus domain model, the traceability check and the function-point count

Five flash-cards · a formative quiz (4 multiple-choice + 2 multiple-select)

Nothing here is graded.

ACTIVE BREAK · 0:56–1:00 · 4 MINUTES

🧘 Zen-break, then 🃏 flash-cards

One quiet minute, then a flash-card sprint on the modeling terms

1. One minute of quiet: stand up, roll your shoulders, look at something far away, and breathe in for four counts and out for six. Nothing to do — on purpose.

2. Three minutes in pairs: partner A reads a term from the slide; partner B defines it in one sentence, without notes. After six terms, swap. The terms: elicitation · functional requirement · non-functional requirement · constraint · Software Requirements Specification · use case · actor · alternative flow · domain model · multiplicity · Class–Responsibility–Collaborator card · traceability · function point.

3. Put a mark next to every term that you could not define: those cards come first in tonight's review.

The answer is in the notes of this slide and in the module.

Studio today: Lab 05 — Block 5

Time	Step	What happens
1:10–1:20	Step 1 · Set up	Copy the template to /docs/05-srs.md; copy the scoping decision of Block 3; name two or three harms of the product; list six to eight user stories.
1:20–1:40	Step 2 · Elicitation round	Teams paired with a different product; six questions (two open, two closed, two probing); interview each other for six minutes each way; three answers and one surprise.
1:40–2:10	Step 3 · **SRS-Lite** (a short Software Requirements Specification)	8–12 **FRs** (functional requirements), exactly 5 **NFRs** (non-functional requirements) — one each for performance, security or privacy, usability, reliability and maintainability — and 3 constraints — each with an **ID** (identifier), a source and a verification line.
2:10–2:30	Step 4 · Use-case model	Actors; a use-case diagram with at least five use cases; two fully-dressed use cases with at least two alternative flows each.
2:30–2:45	Step 5 · Domain model	**CRC** (Class–Responsibility–Collaborator) round with index cards; 5–8 analysis classes with attributes, associations and multiplicities.
2:45–2:50	Step 6 · First function-point count	Tick each element of the **FR** (functional requirement) list as **EI** (External Input), **EO** (External Output), **EQ** (External Inquiry), **ILF** (Internal Logical File) or **EIF** (External Interface File); **UFP** (Unadjusted Function Points) and **FP** (Function Points) with ΣF = 35.
2:50–3:00	Wrap-up	The Block 5 box and the deadline; exit ticket: “One requirement you rewrote to make it testable — before and after.”

Home Sprint and next week

- Finish Block 5 — Steps 7 to 11, one hour at most
- Recommended: tag block5-draft by Sunday, October 25, 23:59 → written feedback before the Mid-Term
- Final version, tag block5: Sunday, December 20, 2026, 23:59
- Read Pressman and Maxim, chapters 7 and 8, and the function-point pages of chapter 25
- Optional, never graded: two function-point exercises in /docs/numbers.md (provided separately)
- Flash-cards: the thirteen terms of the week
- Next: Class 6, Tuesday, October 27 — Software Design: Architecture, Patterns & Frameworks

Links (1/2): legal, safety, standards and bodies of knowledge

Course, legal and safety

Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0)

VirusTotal — address scanner

VirusTotal — file scanner

Google Transparency Report — Safe Browsing site status

Requirements engineering — standards and bodies of knowledge

ISO/IEC/IEEE 29148 (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 29148), 2018 edition — Requirements engineering

Draft of the next edition of 29148 (ballot stage, 2026)

IEEE Std 830 (Institute of Electrical and Electronics Engineers Standard 830)-1998 — Recommended Practice for Software Requirements Specifications

ISO/IEC 25010 (International Organization for Standardization / International Electrotechnical Commission standard 25010), 2023 edition — Product quality model

SWEBOK (Software Engineering Body of Knowledge) Guide, version 4.0 (**IEEE** (Institute of Electrical and Electronics Engineers) Computer Society)

BABOK Guide (A Guide to the Business Analysis Body of Knowledge) — the **IIBA** (International Institute of Business Analysis)

IREB (International Requirements Engineering Board) — Certified Professional for Requirements Engineering, Foundation Level

Christel and Kang (1992), Issues in Requirements Elicitation — **SEI** (Software Engineering Institute)

Links (2/2): methods, Canada and Québec, textbook and tools

Writing and modelling requirements

EARS (Easy Approach to Requirements Syntax) — Alistair Mavin

Volere Requirements Specification Template

Alistair Cockburn

Jacobson (1987), Object-oriented development in an industrial environment

OMG (Object Management Group) — **UML** (Unified Modeling Language) 2.5.1 specification

Beck and Cunningham (1989), A Laboratory for Teaching Object-Oriented Thinking

ISO/IEC 20926 (International Organization for Standardization / International Electrotechnical Commission standard 20926), 2009 edition — the function-point counting method

W3C (World Wide Web Consortium) — **WCAG** (Web Content Accessibility Guidelines) 2.2

Canada and Québec: one audit and the privacy rules

Office of the Auditor General of Canada — Report 1, Building and Implementing the Phoenix Pay System (2018)

Commission d'accès à l'information du Québec

Bill 64 (Law 25) at the National Assembly of Québec

Act respecting the protection of personal information in the private sector (Légis Québec)

Office of the Privacy Commissioner of Canada

Textbook and the Block 5 tools

Pressman and Maxim, Software Engineering: A Practitioner's Approach, 9th edition (McGraw Hill)

Mermaid — class diagrams in text

Mermaid — use-case diagrams (new in version 12)

Mermaid Live Editor

GitHub Docs — Creating diagrams

draw.io

PlantUML

Pro Git — Tagging

Python

Legal notice, credits and sources

Your use of these slides is governed by the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (version 2.0).

Licence and copyright (section 4) · third-party content (4.6) · originality (4.7) · no affiliation (5) · fictional data only (6.7, 18.2) · Code of Ethics (9) · your own equipment (10) · link and download safety (14, Annex C) · no legal advice (16) · no recording of classmates (17) · academic integrity (18). The acknowledgement form (Annex A, e-signature) is provided separately.

Check every link with VirusTotal before you open it.

Concepts after Pressman and Maxim, 9th edition, chapters 7, 8 and 25; **ISO/IEC/IEEE 29148** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers standard 29148) (2018); **ISO/IEC 25010** (International Organization for Standardization / International Electrotechnical Commission standard 25010) (2023); Cockburn (2001); Jacobson (1987); Beck and Cunningham (1989); Christel and Kang (1992); Albrecht (1979).

© 2026 Cyber.SoHo — Educational Hub. All rights reserved. Educational content only. Use of this module and its content is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy. No affiliation with any vendor, standards body, platform or institution. “From the Industry to the Classroom — Building the Next Generation of IT Pros.”

© 2026 | From the Industry to the Classroom — Building the Next Generation of IT Pros. ALL Rights Reserverd.



Cyber.SoHo Educational Hub